MODULE 19 · DOMAIN 2 - SECURITY

Secrets, Parameters and Configuration

Secrets Manager, Parameter Store and AppConfig – three stores, and the clause in each scenario that picks one.

DOMAIN 2 of 4 EXAM WEIGHT 26% MODULE 19 of 34

admin@sthithapragna.com

A secret and a configuration value look identical. What separates them is whether it **rotates**.

Where secrets must never live

- **In source code.** It reaches every clone, every fork and the whole commit history.
- **In a container image.** Anyone who can pull the image can read the layers.
- **In a Lambda environment variable.** Readable by anyone with GetFunctionConfiguration.
- **In a CloudFormation template parameter** without NoEcho, where it appears in the stack events.
- Rotating a leaked secret is the fix. Deleting the commit is not, because the history remains.

> ON THE EXAM
>
> Environment variables are the subtle one. They are fine for a table name or an endpoint, and wrong for anything that grants access.

AWS Secrets Manager

- Stores a string or binary secret up to **64 KB**, encrypted with a KMS key.
- Retrieved with **GetSecretValue**, which returns the current value by default.
- Supports **automatic rotation**, which is the feature that justifies its cost.
- Has a **resource policy**, so it can be shared across accounts.
- Can be **replicated to other Regions**, staying in sync with the primary.

> ON THE EXAM
>
> About 40 cents per secret per month plus API calls. Trivial for fifty database passwords, and significant for five thousand configuration values.

Sharing a secret

- A secret has a **resource policy**, so another account can be named as a principal.
- The other account also needs **kms:Decrypt** on the key. Module 18's rule applies here.
- The default AWS managed key **cannot** be shared, so cross-account sharing needs a customer managed key.
- **Replication** keeps read-only copies in other Regions, updated when the primary changes.
- A replica can be **promoted** to standalone if the primary Region is lost.

> ON THE EXAM
>
> The AWS managed key restriction is the practical trap: cross-account sharing fails until the secret is re-encrypted under a customer managed key.

Staging labels

- **AWSCURRENT** is the live value. This is what GetSecretValue returns by default.
- **AWSPENDING** is a new candidate value during rotation, not yet live.
- **AWSPREVIOUS** is the last known-good value, kept for rollback.
- A label attaches to exactly one version. Moving AWSCURRENT moves AWSPREVIOUS automatically.
- Applications should always read the default label and never pin to a version ID.

> ON THE EXAM
>
> Labels are why an application never sees a half-rotated secret: the new value is staged as AWSPENDING and only becomes AWSCURRENT once it has been tested.

The rotation lifecycle

01

createSecret

Generate a new value and store it as AWSPENDING.

02

setSecret

Update the target system to accept the new credential.

03

testSecret

Verify the new credential actually works.

04

finishSecret

Promote AWSPENDING to AWSCURRENT. The old value becomes AWSPREVIOUS.

ON THE EXAM

Four steps a rotation Lambda must implement. testSecret is the one that matters: promotion only happens after the new credential has been proven to work.

Managed and custom rotation

- **Managed rotation** needs no Lambda at all. AWS runs it for you.
- Covers RDS, Aurora, DocumentDB and Redshift admin credentials, among others.
- **Rotation by Lambda** uses an AWS-provided template or your own function.
- Templates exist for the common databases. Anything else means writing the four steps.
- The rotation function needs network access to the target, so a private database means a VPC.

> ON THE EXAM
>
> When a scenario says 'rotate an RDS password with no code', managed rotation is the answer, and it genuinely involves writing nothing.

Single and alternating users

- **Single user** rotates the password of one account, in place.
- There is a brief window where an application holding the old password fails.
- **Alternating users** keeps two accounts and switches between them each rotation.
- The old credential stays valid until the next rotation, so there is no failure window.
- Alternating needs a superuser credential to manage the second account.

> ON THE EXAM
>
> Alternating users is the answer when a scenario says rotation must cause no downtime and applications may cache credentials.

Systems Manager Parameter Store

- Hierarchical key-value storage. Keys look like **/myapp/prod/db/host**.
- Types: **String**, **StringList** and **SecureString**, which is KMS-encrypted.
- **GetParametersByPath** fetches an entire branch in one call, which suits per-environment config.
- Parameters are **versioned**, and labels can pin a release to a known version.
- **No built-in rotation.** That is the dividing line with Secrets Manager.

> ON THE EXAM
>
> The hierarchy is the practical advantage. One GetParametersByPath call on /myapp/prod/ returns every setting an environment needs.

Standard and advanced tiers

	STANDARD AGAINST ADVANCED
Value size	4 KB against 8 KB
Parameters per Region	10,000 against 100,000
Cost	Free against about 5 cents per parameter per month
Parameter policies	Not available against available
Cross-account sharing	Not available against available

ON THE EXAM

Tier is not a security setting. A parameter is a SecureString, or it is not, entirely independently of which tier it sits in.

Parameter policies

- **Expiration** deletes a parameter automatically at a given time.
- **ExpirationNotification** sends an EventBridge event ahead of that expiry.
- **NoChangeNotification** fires when a parameter has not been updated for a period.
- Together these give a poor-man's rotation reminder for values Parameter Store cannot rotate.
- They notify. They do not rotate anything. That distinction is the exam point.

> ON THE EXAM
>
> A no-change notification tells you a credential has gone stale. Secrets Manager would simply have rotated it. That gap is the reason Secrets Manager exists.

Parameter Store and Secrets Manager together

- Parameter Store can **reference** a Secrets Manager secret, using a /aws/reference/secretsmanager/ path.
- So one GetParametersByPath call can return configuration and a rotating secret together.
- The secret still lives in Secrets Manager, so rotation and sharing still work.
- The caller needs permission on **both** services for the reference to resolve.
- This keeps application code reading from one place, whatever the value actually is.

> ON THE EXAM
>
> A useful pattern: rotating credentials stay in Secrets Manager where they belong, while the application still fetches everything from one hierarchy.

Secrets Manager or Parameter Store?

SECRETS MANAGER

Rotation and sharing

- Automatic rotation, managed or custom
- Cross-account resource policy
- Cross-Region replication
- About 40 cents per secret per month
- Purpose-built for credentials

PARAMETER STORE

Configuration at scale

- No rotation at all
- Standard tier is free
- Hierarchical paths and bulk fetch
- Holds configuration and secrets together
- Advanced tier adds policies and sharing

THE TIEBREAKER

Rotation is the fork. If a value must rotate, or be shared across accounts or Regions, it is Secrets Manager. Everything else belongs in Parameter Store.

Reading secrets efficiently

- Calling GetSecretValue on every request adds latency, cost and throttling risk.
- Fetch at **startup**, outside the Lambda handler, so it happens once per environment.
- The **Parameters and Secrets Lambda extension** caches values behind a localhost endpoint.
- Its cache TTL is configurable, so rotated values are still picked up.
- AWS also publishes caching client libraries for several languages.

> ON THE EXAM
>
> Module 3's scenario 4 was exactly this. Caching with a TTL shorter than the rotation period is the pattern that satisfies both latency and freshness.

Injecting secrets into compute

- **ECS task definitions** have a **secrets** field that injects values as environment variables at start.
- The **task execution role** fetches them, so the value never appears in the task definition.
- **CloudFormation dynamic references** resolve a secret at deploy time without storing it in the template.
- Written as **{{resolve:secretsmanager:...}}** or **{{resolve:ssm-secure:...}}**.
- A resolved value can still leak into logs or drift detection. Reference, do not embed.

> ON THE EXAM
>
> The ECS secrets field is the right answer for containers, and it is a different thing from putting the value in the plain environment block.

AWS AppConfig

- Part of Systems Manager. Built for configuration that changes **while the application runs**.
- **Feature flags** and operational settings, toggled without a deployment.
- **Validators** check a configuration before it is released. A JSON Schema, or a Lambda function.
- Configuration is stored in Parameter Store, S3, or AppConfig's own hosted store.
- It is not a secret store. Secrets still belong in Secrets Manager or a SecureString.

> ON THE EXAM
>
> The distinguishing capability is validation and controlled release. Parameter Store changes a value instantly, everywhere, with nothing checking it first.

AppConfig deployment strategies

- A strategy defines **growth rate**, **growth type**, **bake time** and final duration.
- Linear or exponential growth, so a change reaches hosts gradually rather than all at once.
- **Bake time** is a monitoring window after full deployment before it is declared successful.
- A **CloudWatch alarm** can trigger an **automatic rollback** during deployment or bake time.
- Predefined strategies exist: AllAtOnce, Linear50PercentEvery30Seconds, Canary10Percent20Minutes.

> ON THE EXAM
>
> Automatic rollback on an alarm is the feature to remember. A bad configuration change is reverted without anyone being paged.

AppConfig or Parameter Store?

PARAMETER STORE

Read a value

- Simple key-value retrieval
- Changes apply instantly, everywhere
- No validation before release
- No gradual rollout
- Free at standard tier

APPCONFIG

Release a change

- Validated before deployment
- Rolled out gradually
- Automatic rollback on an alarm
- Built for feature flags
- Charged per configuration request

THE TIEBREAKER

Ask whether a bad value would cause an incident. If yes, the validation and rollback are worth paying for. If not, Parameter Store is simpler.

When secrets leak

- **Rotate first.** A leaked secret is compromised the moment it leaves your control.
- Removing the commit does not help. The value exists in every clone and every fork.
- **Amazon Q Developer** and **CodeGuru** scan for hard-coded credentials in code review.
- **Amazon Macie** discovers credentials and personal data already sitting in S3.
- **GuardDuty** alerts on credentials being used from an unexpected location.

> ON THE EXAM
>
> The ordering matters: rotate, then investigate. Cleaning up the repository is housekeeping, not remediation.

Configuration in the deployment pipeline

- CodeBuild reads secrets through **secrets-manager** and **parameter-store** blocks in the buildspec.
- Those resolve at build time, so the value never sits in the buildspec file.
- CodeBuild **masks** known secret values in logs, but only ones it resolved itself.
- A secret echoed by your own script can still reach the build log in plain text.
- Give each environment its own secret, and the pipeline role access to only what it deploys.

> ON THE EXAM
>
> Build logs are a common leak path and are often more widely readable than the secrets themselves. Never echo a resolved value.

Where does this value belong?

THE SCENARIO SAYS	THE INTENDED ANSWER
A database password that must rotate	Secrets Manager, managed rotation
Thousands of config values, cost-sensitive	Parameter Store standard tier
A static API key, one account, no rotation	Parameter Store SecureString
A secret shared across accounts or Regions	Secrets Manager
A feature flag toggled without deploying	AWS AppConfig
A config change that must roll back on an alarm	AWS AppConfig
An endpoint URL or a table name	Environment variable or Parameter Store

ON THE EXAM

Worth screenshotting. Rotation, sharing and controlled release are the three capabilities that pull a value out of the cheapest option.

Ask first: does this value need to rotate, to be shared, or to be released carefully?

> Compliance requires the master password of an Amazon RDS for PostgreSQL database to change every 30 days. Several applications connect to it, and none may fail during the change.
>
> The team wants to write and maintain as little code as possible, and applications should not need modifying to handle the change.

How should the rotation be implemented?

A. A scheduled Lambda function that changes the password and updates a Parameter Store SecureString

B. Secrets Manager with managed rotation on a 30-day schedule, using the alternating users strategy

C. A Parameter Store advanced parameter with a no-change notification after 30 days

D. Rotate manually every month and redeploy the applications with the new password

◆ PAUSE AND DECIDE

How should the rotation be implemented?

A Scheduled Lambda plus Parameter Store

Rebuilding rotation by hand, without staging labels or a test step. A failed run leaves a changed password and a stale parameter.

B Managed rotation, alternating users

RDS is covered by managed rotation, so no code at all. Alternating users keeps the previous credential valid, so nothing fails mid-rotation.

C No-change notification

Notifies that the password is stale. It does not change it, which is the entire requirement.

D Manual rotation plus redeploy

A monthly deployment purely to change a password, with downtime and a manual step that will eventually be forgotten.

THE SIGNAL

Three clauses map to three features: RDS means managed rotation, 'none may fail' means alternating users, and 'as little code as possible' rules out A.

> A platform team manages about 3,000 configuration values across dev, staging and production: endpoint URLs, timeouts, feature toggles, queue names and a handful of static third-party API keys. None of them rotates.
>
> Applications should fetch all settings for their environment in one call. Cost matters, and the current approach of one Secrets Manager secret per value costs over 1,200 dollars a year.

Where should these values live?

A. Keep Secrets Manager, but consolidate values into fewer JSON secrets

B. Environment variables set at deployment time by the CI/CD pipeline

C. Parameter Store advanced tier, for the higher parameter limit

D. Parameter Store standard tier, using a path hierarchy per environment, with SecureString for the API keys

◆ PAUSE AND DECIDE

Where should these values live?

A Fewer, larger secrets

Reduces cost by making configuration harder to read and change. Updating one timeout means rewriting a large JSON blob.

B Environment variables from CI/CD

Changing any value now requires a deployment, and the API keys end up readable in the console.

C Advanced tier

Advanced costs about 5 cents per parameter per month, so 3,000 parameters is roughly 1,800 dollars a year. Worse than the problem.

D Standard tier with hierarchy

3,000 is within the 10,000 standard limit, standard is free, GetParametersByPath serves the one-call requirement, and SecureString covers the API keys.

THE SIGNAL

Option C is the trap: 'more parameters' sounds like it needs the higher tier, but 3,000 is well inside the standard limit of 10,000.

A team wants to release a new checkout algorithm to 5% of traffic, watch error rates, and expand gradually. If errors rise, the change must revert automatically without a deployment or a human decision.

The toggle is read by a fleet of ECS tasks. A malformed value has previously taken the checkout page down.

What should the team use?

A AWS AppConfig, with a validator, a canary deployment strategy and a CloudWatch alarm for rollback

B A Parameter Store parameter that tasks poll every 30 seconds

C A Secrets Manager secret holding the flag, with rotation disabled

D An environment variable, redeploying the service to change the percentage

◆ PAUSE AND DECIDE

What should the team use?

A AppConfig with validator and alarm

Gradual rollout, validation before release, and automatic rollback on an alarm. All three requirements come from one service.

B Polled Parameter Store value

The closest wrong answer. It changes the value without a deployment, and offers no validation, no gradual rollout and no automatic rollback.

C Secrets Manager

A feature flag is not a secret. It costs more, and adds none of the release-control features the scenario needs.

D Environment variable

Requires a deployment for every percentage change, which is exactly what the team is trying to avoid.

THE SIGNAL

Three requirements, and only AppConfig has all three. The 'malformed value took checkout down' sentence is what makes validation non-optional.

A Lambda function reads a database credential from Secrets Manager at the top of its handler. It is invoked about 600 times per second at peak. The team sees roughly 50 ms of added latency on every request and occasional ThrottlingException errors from Secrets Manager.

The credential rotates every 7 days. A rotated credential must be picked up without redeploying the function.

What is the best change?

A. Move the secret into a Lambda environment variable encrypted with a customer managed KMS key

B. Request a Secrets Manager quota increase and add exponential backoff

C. Add the Parameters and Secrets Lambda extension, reading from its local endpoint with a cache TTL well under 7 days

D. Move the GetSecretValue call outside the handler, into module initialisation

◆ PAUSE AND DECIDE

What is the best change?

A Encrypted environment variable

Readable by anyone with GetFunctionConfiguration, and frozen at deploy time, so a rotated credential would never be picked up.

B Quota increase and backoff

Pays for 600 identical calls per second rather than removing them. Backoff manages the symptom and adds latency of its own.

C Parameters and Secrets extension

Removes the API call from the request path entirely, and the TTL guarantees a rotated credential is collected without a deployment.

D Fetch during initialisation

The strongest wrong answer. It fixes latency and throttling, and a warm environment can live for hours, so a rotated credential may be missed for a long time.

THE SIGNAL

Compare with module 3's scenario 4, which was the same shape. Option D is right for a static secret and wrong here, because rotation is the added constraint.

What to carry forward

- **Rotation is the fork.** Rotating, shared or replicated means Secrets Manager. Everything else, Parameter Store.
- **AWSCURRENT, AWSPENDING, AWSPREVIOUS**, and a rotation function's four steps.
- **Alternating users** when rotation must cause no failure window.
- Parameter Store tier is capacity, not security. SecureString is independent of tier.
- **AppConfig** when a configuration change needs validation, gradual rollout or automatic rollback.

> ON THE EXAM
>
> That completes Domain 2. Next: module 20 opens Domain 3 with CloudFormation, and the deployment domain.